

Code in mehreren Sprachen schreiben

Databricks-Notebooks erlauben Python, SQL, Scala und R – sogar gemischt innerhalb desselben Notebooks. Diese Seite erklärt die Werkzeuge dafür: Formatierung, Sprachwechsel per "Magic Commands" und wichtige Zusatzfunktionen.

Voraussetzung: aktive Compute-Sitzung

Funktionen wie Autovervollständigung, automatische Python-Formatierung und der Debugger benötigen eine aktive Rechenressource (Compute), an die das Notebook angehängt ist.

Code in Workspace-Dateien organisieren

Ab Databricks Runtime 11.3 LTS lassen sich Quellcode-Dateien direkt im Workspace anlegen und in Notebooks importieren. Das ermöglicht saubere Code-Organisation und Wiederverwendung über mehrere Notebooks hinweg, statt allen Code in einem einzigen Notebook zu bündeln.

Code automatisch formatieren

Python

Für Python ist der Formatierer **Black** integriert (ab Databricks Runtime 11.3 LTS vorinstalliert, zusammen mit `tokenize-rt`). Bei älteren Runtimes muss man die Bibliotheken manuell installieren. Black setzt den Python-Styleguide **PEP 8** um; die Einrückung (4 Leerzeichen) ist dabei nicht konfigurierbar.

SQL

Für SQL-Zellen gibt es eine eigene, eingebaute Formatierung.

Formatierung auslösen

- Einzelne Zelle: Tastenkürzel **Strg+Umschalt+F** oder Rechtsklick-Menü

- Mehrere Zellen: markieren, dann **Edit** → **Format Cell(s)**
- Ganzes Notebook: **Edit** → **Format Notebook**

Sprache pro Notebook und pro Zelle

Jedes Notebook hat eine Standardsprache, die unter dem Titel angezeigt wird und über ein Dropdown-Menü änderbar ist. Einzelne Zellen können mit einem sogenannten **Magic Command** (Vorstellen von `%sprache`) davon abweichen.

Wichtige Magic Commands für Sprachen

Befehl	Wirkung
<code>%python</code>	Führt die Zelle als Python-Code aus
<code>%sql</code>	Führt die Zelle als SQL-Abfrage aus
<code>%scala</code>	Führt die Zelle als Scala-Code aus
<code>%r</code>	Führt die Zelle als R-Code aus

Wichtig: Variablen, die in einer Sprache definiert wurden, sind in einer anderen Sprache **nicht** automatisch verfügbar. Jede Sprache läuft in einem eigenen Interpreter (REPL); Daten können nur über externe Ressourcen wie Tabellen oder Dateien ausgetauscht werden.

Weitere nützliche Magic Commands

Befehl	Bedeutung
<code>%sh</code>	Shell-Befehl, läuft nur auf dem Treiber-Knoten (Driver)
<code>%fs</code>	Kurzform für Dateisystem-Operationen (dbutils)
<code>%md</code>	Rendert die Zelle als Markdown (Text, Bilder, Formeln, LaTeX)
<code>%pip</code>	Installiert Python-Pakete, nur für das aktuelle Notebook gültig
<code>%run</code>	Führt ein anderes Notebook aus und übernimmt dessen Funktionen/Variablen

<code>%tensorboard</code>	Zeigt TensorBoard-Oberfläche an (nur ML-Runtime)
<code>%skip</code>	Überspringt die Ausführung dieser Zelle
<code>%uv pip</code>	Schnellere Paketverwaltung mittels "uv" statt klassischem pip

SQL innerhalb von Python

SQL-Syntaxhervorhebung und Autovervollständigung funktionieren sogar innerhalb von Python-Aufrufen wie `spark.sql()`.

Die spezielle Variable `_sqldf`

Das Ergebnis einer SQL-Zelle wird automatisch in der Variable `_sqldf` gespeichert und kann in nachfolgenden Python- oder SQL-Zellen weiterverwendet werden. Einschränkungen: nicht verfügbar auf SQL-Warehouses, nicht bei paralleler Ausführung, und der Wert wird bei jeder neuen SQL-Zelle überschrieben. Wer das Ergebnis behalten möchte, sollte es vor der nächsten SQL-Zelle in eine eigene Variable kopieren.

Parallele SQL-Ausführung

Über den Button "Run now" lassen sich SQL-Zellen auch parallel in eigenen Sitzungen ausführen. Dabei stehen temporäre Views, benutzerdefinierte Funktionen (UDFs), `_sqldf` sowie Standard-Katalog/-Schema nicht zur Verfügung – Tabellen müssen vollständig als `katalog.schema.tabelle` angesprochen werden.

Zusätzliche Funktionen

- **Genie Code:** KI-gestützte Code-Erzeugung, -Erklärung und -Fehlerbehebung direkt im Editor.
- **Python-Debugger:** interaktives, schrittweises Debuggen von Python-Code.
- **IPython Automagic:** Ist standardmäßig aktiviert, sodass Befehle wie `pip` auch ohne das `%`-Zeichen funktionieren.

Einschränkungen im Überblick

- Pro Zelle ist nur ein Magic Command zulässig, und er muss am Anfang der Zelle stehen.
- `%run` muss eine eigene Zelle für sich alleine belegen.
- Auf Databricks Runtime 12.2 LTS und älter müssen `%pip`-Befehle am Anfang des Notebooks stehen.
- Jede Sprache hat ihren eigenen isolierten Interpreter – kein automatischer Variablenaustausch zwischen Sprachen.

Quelle: <https://docs.databricks.com/aws/en/notebooks/notebooks-code>

Stand: 2026-08-06